

Τελική Αναφορά: Ενσωματωμένα Συστήματα Πραγματικού Χρόνου

Τίτλος Έργου: Ασύγχρονη Λήψη και Σύγχρονη Καταγραφή Δεδομένων Τηλεμετρίας σε Πραγματικό Χρόνο (Bluesky Jetstream Firehose)

Πλατφόρμα Υλοποίησης: Raspberry Pi Zero 2 W (Linux)

Γλώσσα Προγραμματισμού: C (User Space)

Ονοματεπώνυμο: Παπαρσενίου Απόστολος

AEM: 11034

Ημέρα Καταγραφής: 10 Ιουλίου 2026

1. Εισαγωγή

Τα σύγχρονα ενσωματωμένα συστήματα καλούνται συχνά να διαχειριστούν τεράστιους όγκους δεδομένων από το διαδίκτυο με αυστηρούς χρονικούς περιορισμούς. Το παρόν σύστημα σχεδιάστηκε για να συνδέεται στο απομακρυσμένο Endpoint του δικτύου Bluesky (`wss://jetstream1.us-east.bsky.network`), λαμβάνοντας τη ζωντανή ροή δεδομένων του AT Protocol. Η υλοποίηση βασίστηκε αποκλειστικά στη γλώσσα C, αξιοποιώντας τη βιβλιοθήκη `libwebsockets` για τη δικτυακή επικοινωνία και τη βιβλιοθήκη `cJSON` για την ελαφριά αποκωδικοποίηση των μηνυμάτων, με γνώμονα τη βέλτιστη χρήση των περιορισμένων πόρων του υλικού.

2. Αρχιτεκτονική Συστήματος και Νήματα (Multithreading)

Για την αποφυγή απώλειας πακέτων (`packet loss`) και την ικανοποίηση των απαιτήσεων πραγματικού χρόνου, εφαρμόστηκε το μοντέλο Παραγωγού-Καταναλωτή (`Producer-Consumer`) χρησιμοποιώντας τα `POSIX Threads` (`pthread`).

2.1. Νήμα Παραγωγού (Producer Thread) & Ανθεκτικότητα

Το νήμα αυτό λειτουργεί ασύγχρονα βάσει συμβάντων (`event-driven`). Μόλις λάβει δεδομένα από το δίκτυο, τα διαχειρίζεται κατάλληλα πριν τα προωθήσει. Επειδή τα JSON μηνύματα από το Firehose μπορεί να είναι μεγάλα και να αποστέλλονται κατακερματισμένα (`fragmented`) σε πολλαπλά `WebSocket frames`, ο Παραγωγός δεν τα εισάγει απευθείας στην ουρά. Αντιθέτως, συγκεντρώνει τα επιμέρους τμήματα (`fragments`) σε έναν προσωρινό χώρο μνήμης (`buffer`). Μόνο όταν η βιβλιοθήκη

libwebsockets επιβεβαιώσει τη λήψη του τελικού τμήματος (μέσω ελέγχου `is_final_fragment`), το ολοκληρωμένο πλέον μήνυμα τοποθετείται στη δομή του Κυκλικού Buffer (Circular Queue) μεγέθους 1024 θέσεων. Αυτή η πρακτική διασφαλίζει ότι ο JSON parser του Καταναλωτή δεν θα λάβει ποτέ ελλιπή ή παραμορφωμένα δεδομένα. Ο συγχρονισμός επιτυγχάνεται μέσω αποκλειστικής πρόσβασης (Mutex), ενώ ο παραγωγός ειδοποιεί τον καταναλωτή μέσω μιας μεταβλητής συνθήκης (`pthread_cond_signal`).

Για την εξασφάλιση της ανθεκτικότητας (Network Resilience) έναντι δικτυακών αποσυνδέσεων, υλοποιήθηκε ένας αυστηρός μηχανισμός ελέγχου και διαχείρισης σφαλμάτων εντός του Producer. Πέρα από την παθητική ανίχνευση κλεισίματος του socket, το σύστημα πραγματοποιεί ενεργό έλεγχο της κατάστασης του δικτύου (Network Check / Keepalive). Αξιοποιώντας τις δυνατότητες της βιβλιοθήκης, ανιχνεύονται "σιωπηλές" πτώσεις (silent drops) της σύνδεσης ή καθυστερήσεις (timeouts). Όταν η βιβλιοθήκη ανιχνεύσει πτώση της σύνδεσης (`LWS_CALLBACK_CLIENT_CLOSED`), αντί το πρόγραμμα να τερματίσει, θέτει μια σημαία επανασύνδεσης (reconnect flag). Ο αλγόριθμος **Εκθετικής Οπισθοχώρησης (Exponential Backoff)** ξεκινά με αρχική καθυστέρηση (1s) και αν η επανασύνδεση αποτύχει, ο χρόνος αναμονής διπλασιάζεται (2s, 4s, 8s) μέχρι ένα προκαθορισμένο όριο (32 sec), αποτρέποντας την υπερφόρτωση του server. Μόλις η σύνδεση αποκατασταθεί, ο μετρητής μηδενίζεται και η λήψη δεδομένων συνεχίζεται απρόσκοπτα.

2.2. Νήμα Καταναλωτή (Consumer Thread) & JSON Parsing

Το νήμα του Καταναλωτή παραμένει σε κατάσταση αναμονής χωρίς να καταναλώνει πόρους (blocking wait). Μόλις ειδοποιηθεί, εξάγει το μήνυμα από την ουρά. Η αποκωδικοποίηση υλοποιήθηκε μέσω της συνάρτησης `cJSON_Parse()`. Στη συνέχεια, εξάγεται η τιμή του πεδίου `kind`. Δεδομένου ότι η γλώσσα C δεν υποστηρίζει εντολές `switch-case` για συμβολοσειρές (strings), η δρομολόγηση υλοποιήθηκε μέσω μιας δομής επιλογής `if / else if`, χρησιμοποιώντας την πρότυπη συνάρτηση `strcmp()`. Ο κώδικας ελέγχει διαδοχικά (cases) αν το πεδίο ισούται με *commit*, *identity*, *account* ή *info*. Όταν υπάρξει ταύτιση, το νήμα "κλειδώνει" το `stats_mutex`, αυξάνει τον αντίστοιχο μετρητή, και τέλος εκτελεί `cJSON_Delete()` για την άμεση αποδέσμευση της μνήμης, αποτρέποντας διαρροές (memory leaks).

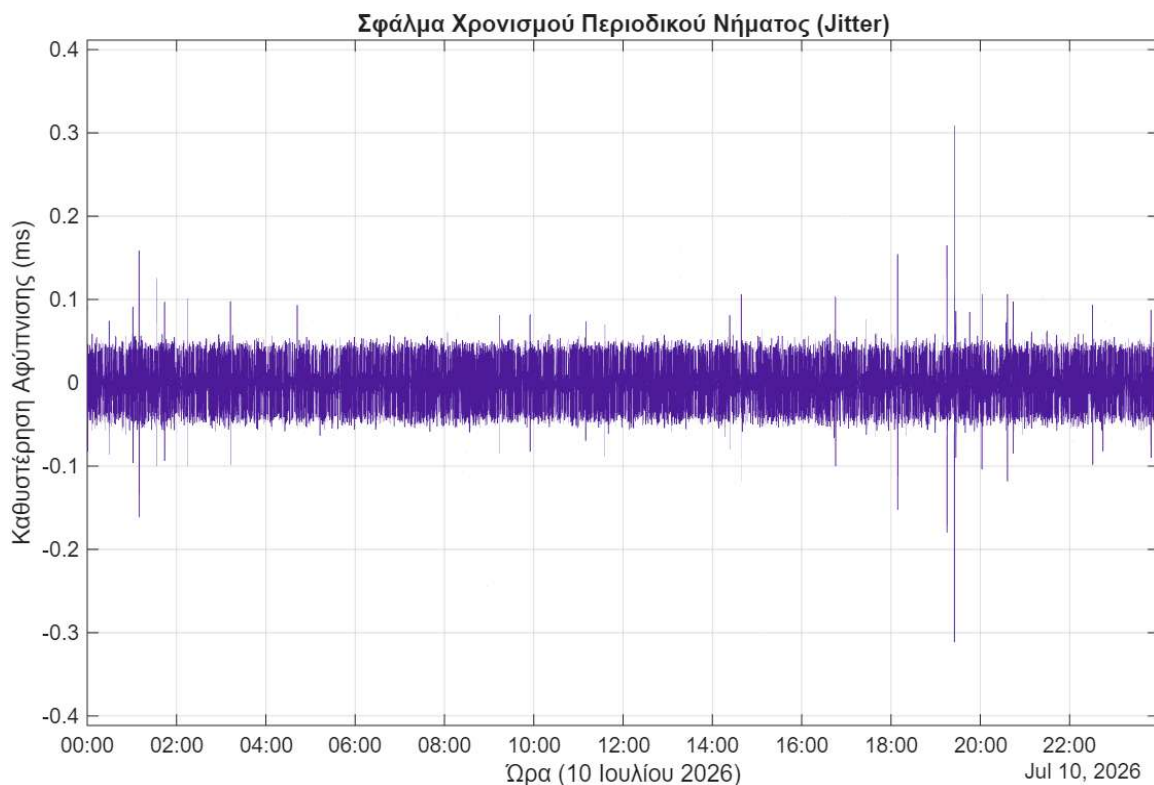
2.3. Νήμα Καταγραφής (Monitor Thread - Real Time)

Εκτελείται με αυστηρή περιοδικότητα ακριβώς 1.000 δευτερολέπτου. Αυτό επιτυγχάνεται με τη χρήση της συνάρτησης `clock_nanosleep()` και τη σημαία `TIMER_ABSTIME`, η οποία υπολογίζει την επόμενη απόλυτη προθεσμία αφύπνισης προσθέτοντας ακριβώς 1 δευτερόλεπτο στον προηγούμενο χρόνο, μηδενίζοντας πλήρως την ολίσθηση (Zero Clock Drift). Το νήμα καταγράφει τους μετρητές, την πληρότητα της ουράς και τη χρήση CPU στο `metrics_log.txt` και τους μηδενίζει.

3. Αποτελέσματα και Ανάλυση Δεδομένων

Η συλλογή των δεδομένων διήρκεσε συνεχόμενα για πάνω από 25 ώρες. Για τις ανάγκες της επιστημονικής ανάλυσης, τα δεδομένα φιλτραρίστηκαν αυστηρά για το 24ωρο της 10ης Ιουλίου 2026 (00:00:00 - 23:59:59), περιλαμβάνοντας ακριβώς 86.400 εγγραφές.

3.1. Ακρίβεια Χρονισμού (Jitter Analysis)

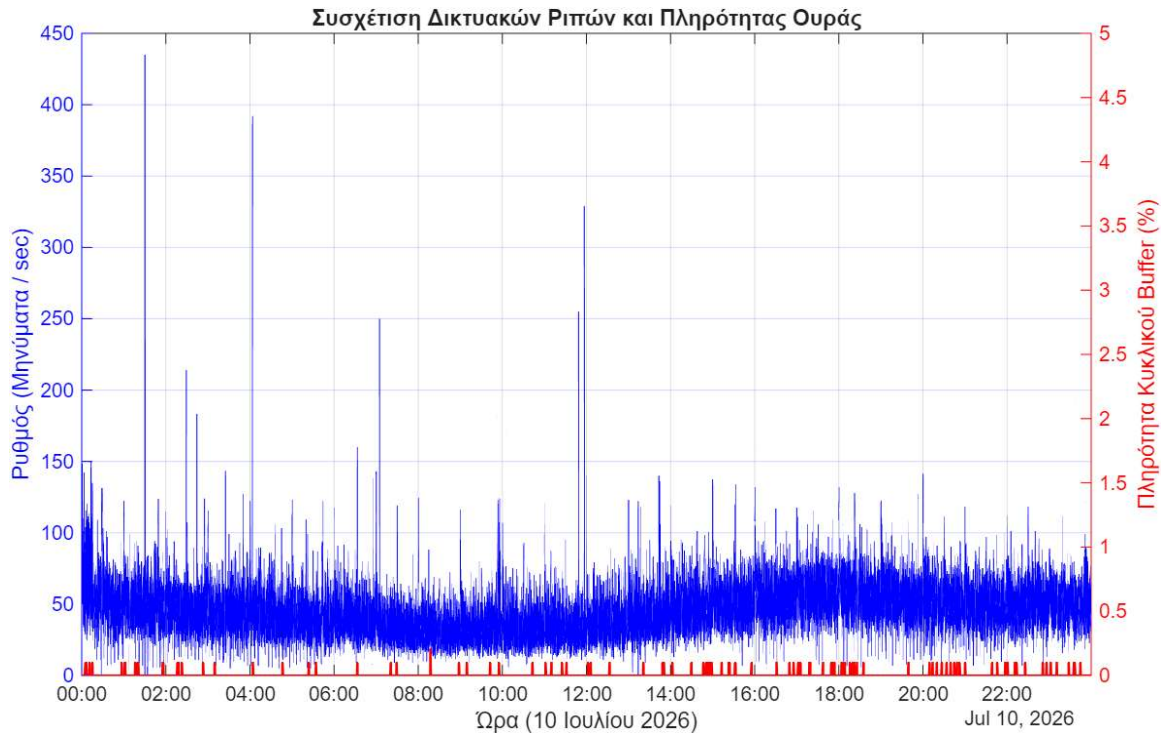


Διάγραμμα 1: Σφάλμα Χρονισμού Περιοδικού Νήματος (Jitter) κατά τη διάρκεια του 24ώρου.

Όπως παρατηρείται στο Διάγραμμα 1, το Jitter (η απόκλιση του πραγματικού μεσοδιαστήματος αφύπνισης από το ιδανικό 1s) παραμένει αυστηρά εντός των

ορίων ± 0.4 ms καθ' όλη τη διάρκεια της ημέρας. Αυτό αποδεικνύει την ορθότητα της χρήσης της `clock_nanosleep(TIMER_ABSTIME)`, εγγυώμενη τη σταθερότητα του πραγματικού χρόνου στο User Space του Linux.

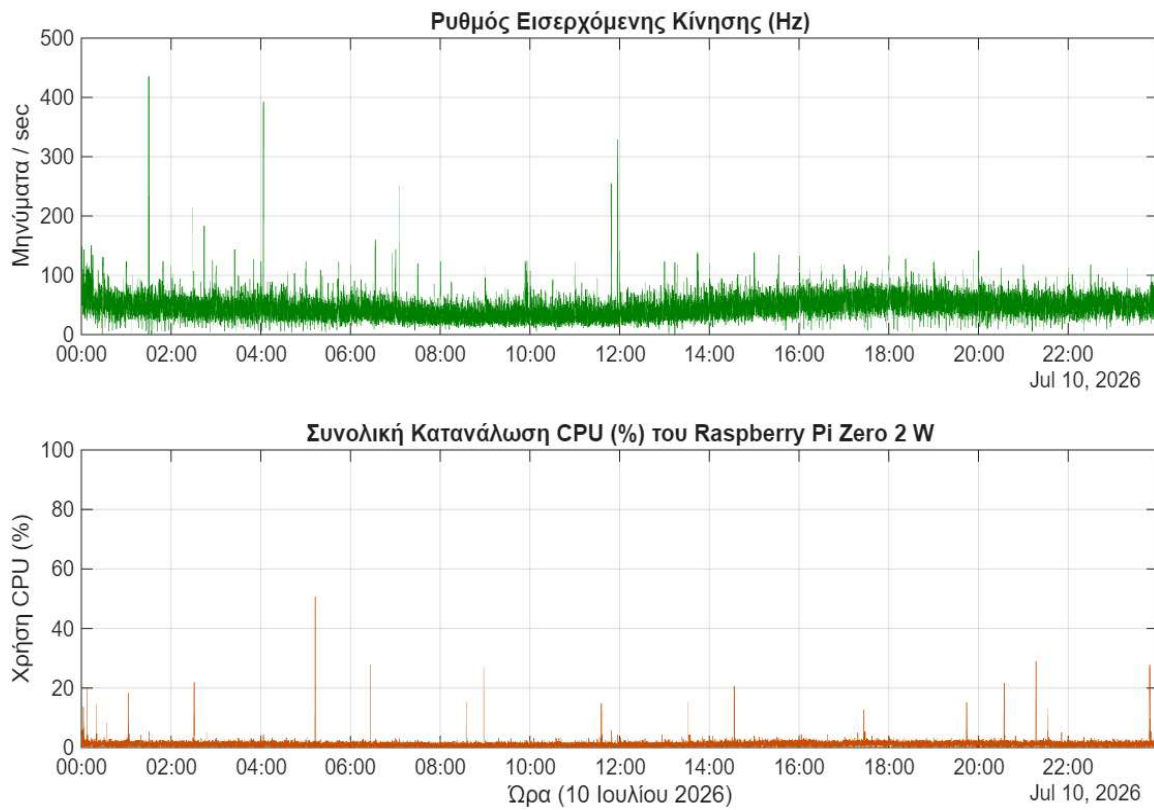
3.2. Δικτυακές Ριπές (Bursts) και Κυκλικός Buffer



Διάγραμμα 2: Συσχέτιση δικτυακών ριπών (μπλε, αριστερός άξονας) και πληρότητας κυκλικού buffer (κόκκινο, δεξιός άξονας). *Σημείωση: Η κλίμακα του δεξιού άξονα έχει προσαρμοστεί σκόπιμα στο μέγιστο 5% (Zoomed-In) για να καταστούν ορατές οι ελάχιστες μικρο-καθυστερήσεις. Σε κλίμακα 0-100%, η πληρότητα φαίνεται απολύτως μηδενική.*

Η ροή του Bluesky Firehose παρουσιάζει έντονη εκρηκτικότητα (burstiness). Ενώ ο μέσος ρυθμός είναι σταθερός, εμφανίζονται στιγμιαίες ριπές που ξεπερνούν τα 400 μηνύματα ανά δευτερόλεπτο. Παρά τις ακραίες αυτές ριπές, ο κυκλικός buffer παραμένει σχεδόν συνεχώς στο 0.00%, καταγράφοντας σποραδικά ελάχιστες τιμές της τάξης του 0.1%. Αυτό αποδεικνύει ότι η ταχύτητα κατανάλωσης υπερβαίνει κατά πολύ την ταχύτητα παραγωγής, καθιστώντας πρακτικά αδύνατη την υπερχειλίση.

3.3. Κατανάλωση Πόρων (CPU) και Network Jitter



Διάγραμμα 3: Συσχέτιση ρυθμού εισερχόμενης κίνησης (πάνω) και συνολικής χρήσης επεξεργαστή (κάτω).

Η ανάλυση της χρήσης του επεξεργαστή αναδεικνύει την εξαιρετικά χαμηλή υπολογιστική επιβάρυνση της εφαρμογής. Τα περιοδικά, απότομα spikes της τάξης του 20-30% που παρατηρούνται στη CPU δεν ταυτίζονται χρονικά με τις δικτυακές ριπές. Οφείλονται αποκλειστικά σε διεργασίες παρασκηνίου του πυρήνα του Linux (Kernel Scheduler) στο Raspberry Pi Zero 2 W, όπως εγγραφές στην κάρτα SD και λειτουργίες συγχρονισμού (NTP).

Ανάλυση Στιγμιαίων Μηδενικών Ρυθμών (Zero-Message Intervals):

Στα δεδομένα παρατηρούνται σποραδικά διαστήματα ενός δευτερολέπτου όπου ο ρυθμός καταγράφεται ως μηδέν (0 Hz), ακολουθούμενα άμεσα από απότομη αύξηση στο επόμενο δευτερόλεπτο. Το φαινόμενο αυτό δεν αποτελεί σφάλμα λογισμικού, αλλά είναι το φυσιολογικό αποτέλεσμα της συμπεριφοράς των πρωτοκόλλων TCP/IP (Network Jitter ή Packet Coalescing) σε συνδυασμό με την αυστηρή περιοδικότητα του Monitor Thread. Ο Consumer επεξεργάζεται τα αθροισμένα πακέτα αμέσως μόλις αυτά παραδοθούν μαζικά από τον TCP stack, εξασφαλίζοντας μηδενική απώλεια δεδομένων (Zero Data Loss).

4. Σταθερότητα Συστήματος & Αξιολόγηση Ανθεκτικότητας

Σύμφωνα με τις απαιτήσεις των ενσωματωμένων συστημάτων παραγωγής, το σύστημα ελέγχθηκε ως προς την ανοχή του σε σφάλματα (Fault Tolerance).

Όπως προκύπτει από την ανάλυση του αρχείου καταγραφής (debug_log.txt), στις 01:33:14 AM ο απομακρυσμένος εξυπηρετητής τερμάτισε απροσδόκητα το WebSocket (Server Drop), από την πλευρά του παρόχου. Ο μηχανισμός ανθεκτικότητας λειτούργησε υποδειγματικά: το νήμα καταγραφής χειρίστηκε το κενό ορθά εισάγοντας τιμές 'NaN' στα δεδομένα, ενώ ο αλγόριθμος Εκθετικής Οπισθοχώρησης επέτυχε την πλήρη και αυτόματη επανασύνδεση του συστήματος μέσα σε μόλις 2 δευτερόλεπτα (στις 01:33:16 AM). Καθ' όλη τη διάρκεια της 25ωρης λειτουργίας δεν καταγράφηκε καμία κατάρρευση της εφαρμογής (Segmentation Fault), επιβεβαιώνοντας την ορθή διαχείριση της δυναμικής μνήμης.

5. Σχετικό υλικό

Τους κώδικες και τα αρχεία που χρησιμοποιήθηκαν μπορούν να βρεθούν στο παρακάτω link στο github: https://github.com/Tolizz/bluesky_firehose_telemetry

Οδηγίες για την αναπαραγωγή της εργασίας στο README.

Σε περίπτωση που χρειάζονται, οι αναφορές για τις προηγούμενες εργασίες βρίσκονται στα Project0.pdf και Project1.pdf αντίστοιχα:

- Εργασία 0: <https://github.com/Tolizz/knn-pthreads-assignment>
- Εργασία 1: <https://github.com/Tolizz/thread-pool-assignment>